NetWatch: A Cybernetic Guardrail Architecture for Multimodal Autonomous Systems

NetWatch Project

September 3, 2026

Abstract

Modern autonomous AI agents face significant security challenges where static safety filters and prompt engineering fail against dynamic runtime vulnerabilities, such as the Agent Runtime Authority Gap, Cache-Augmented Generation (CAG) poisoning, and GraphRAG topological attacks. This thesis presents **NetWatch**, a cybernetic guardrail architecture that balances applied mathematics (continuous Control Barrier Functions, Mahalanobis distance outlier detection, Fast Fourier Transforms, and Differential Privacy) with robust software engineering (tri-hook runtime API interception middleware, sandboxing, and provenance tracking).

Keywords: AI Agent Security, Runtime Application Self-Protection (RASP), Control Barrier Functions, Mahalanobis Distance, GraphRAG Defense, Multimodal Guardrails.

Contents

Abstract **1**

1 Introduction **4**
1.1 Background and Motivation 4
1.2 Problem Statement 4
1.3 Research Questions 4
1.4 Thesis Contributions 5
1.5 Thesis Organization 5

2 Related Work **6**
2.1 Vulnerabilities in Autonomous Agent Runtimes 6
2.2 GraphRAG and Vector Knowledge Base Security 6
2.3 Mathematical Verification and Safe Execution Boundaries 6
2.4 Runtime Application Self-Protection (RASP) for AI 6

3 Methodology **7**
3.1 System Architecture Overview 7
3.2 The Tri-Hook Defense Framework 7
3.2.1 1. Pre-Retrieval Hook (Perception & Memory) 7
3.2.2 2. Pre-Execution Hook (Action & API Gateway) 7
3.2.3 3. Post-Execution Hook (Context & Exfiltration Prevention) . . . 7

4 Experiments **9**
4.1 Experimental Setup and Testbed 9
4.2 Threat Models and Attack Scenarios 9
4.3 Evaluation Datasets and Red-Teaming Benchmarks 9
4.4 Baseline Configurations 9

5 Evaluation **10**
5.1 Detection Efficacy (RQ1 & RQ2) 10
5.2 Safety Barrier Enforcement (RQ3) 10
5.3 Cybernetic Rate Limiting Performance (RQ4) 10
5.4 System Overhead and Latency Analysis 10

6 Discussion **11**
6.1 Balancing Mathematical Rigor and Software Practicality 11
6.2 Security Depth vs. Latency Overhead 11
6.3 Assumptions and Limitations 11

7 Future Work **12**
7.1 Adaptive Latent Space Barrier Learning 12
7.2 Hardware-Accelerated Cryptographic Hooks 12
7.3 Multi-Agent Swarm Defense Protocols 12

AI Disclosure **13**

References **14**

A Mathematical Proofs **15**
A.1 Forward Invariance of Control Barrier Functions 15

B Architecture Configuration and Schemas **16**
B.1 Middleware Configuration Schema 16

Chapter 1

Introduction

1.1 Background and Motivation

The proliferation of autonomous AI agents operating in production environments has highlighted critical vulnerabilities in existing safety paradigms. Unlike traditional large language model (LLM) deployments that rely primarily on prompt filtering and lexical heuristics, agentic systems possess tool-use capabilities, execution privileges, and persistent memory.

1.2 Problem Statement

Static safety filters are insufficient against dynamic attacks targeting the Agent Runtime Authority Gap. Attackers exploit working memory (CAG poisoning), topological dependencies (GraphRAG relation injection), and visual steganography to hijack agent execution pipelines.

1.3 Research Questions

- **RQ1 (GraphRAG Topological Defense):** How can graph-theoretic anomaly detection algorithms be integrated into an asynchronous ingestion middleware to mathematically isolate adversarial relation injections before corrupting multi-hop reasoning?
- **RQ2 (Vision-Language Sandboxing):** To what extent can frequency domain analysis and Fourier transforms in an isolated vision-proxy gateway neutralize steganographic prompt injections in Vision-Language Models (VLMs) without degrading processing latency?
- **RQ3 (Latent Barrier Steering):** How effectively can continuous Control Barrier Functions (CBFs) be mapped onto discrete API action-spaces to guarantee zero-violation execution during multi-step tool hijacking attacks?
- **RQ4 (Cybernetic Rate Limiting):** How can Proportional-Integral-Derivative (PID) control theory be synthesized with dynamic Risk Token-Buckets in the API gateway to throttle network bandwidth prior to data exfiltration?

1.4 Thesis Contributions

- Formulation of a balanced applied mathematics and software engineering Tri-Hook Runtime Architecture.
- Design and implementation of real-time pre-retrieval, pre-execution, and post-execution interception hooks.
- Comprehensive empirical evaluation against standardized agent red-teaming benchmarks.

1.5 Thesis Organization

The remainder of this thesis is organized as follows: Chapter 2 reviews related literature; Chapter 3 describes the mathematical and software architecture of NetWatch; Chapter 4 details the experimental setup; Chapter 5 presents empirical findings; Chapter 6 discusses implications and limitations; Chapter 7 outlines future research avenues; and the Appendices provide detailed mathematical proofs and configuration schemas.

Chapter 2

Related Work

2.1 Vulnerabilities in Autonomous Agent Runtimes

Recent advances in autonomous agents have revealed attack vectors such as Indirect Prompt Injection (IPI), Cache-Augmented Generation (CAG) poisoning, and Confused Deputy attacks where agents are tricked into misusing internal credentials.

2.2 GraphRAG and Vector Knowledge Base Security

Attacks exploiting vector databases and graph-based retrieval augmented generation systems via topological perturbation, competing relation injection, and embedding space poisoning.

2.3 Mathematical Verification and Safe Execution Boundaries

Constrained Markov Decision Processes (CMDP) and Control Barrier Functions (CBFs) used in robotic systems, and their emerging application to discrete LLM agent action spaces.

2.4 Runtime Application Self-Protection (RASP) for AI

Zero-Trust architectures for Non-Human Identities (NHI), ephemeral credential issuance, and dynamic API gateway middleware.

Chapter 3

Methodology

3.1 System Architecture Overview

The NetWatch architecture operates as an inline, deterministic gateway between the model's latent representation space and external software execution environments.

3.2 The Tri-Hook Defense Framework

3.2.1 1. Pre-Retrieval Hook (Perception & Memory)

- **Software Engineering:** Vector DB sandboxing and proxy ingestion pipelines.
- **Applied Mathematics:** High-dimensional Mahalanobis distance outlier detection:

$$D_M(x) = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)} \tag{3.1}$$

  and 2D Fast Fourier Transform (FFT) spatial frequency analysis for visual inputs to detect steganographic spikes.

3.2.2 2. Pre-Execution Hook (Action & API Gateway)

- **Software Engineering:** Dual-agent isolation pattern and RASP middleware parameter freezing.
- **Applied Mathematics:** Continuous Control Barrier Functions (CBF):

$$h(x) \geq 0, \quad \dot{h}(x) + \alpha(h(x)) \geq 0 \tag{3.2}$$

  guaranteeing forward invariance within safe execution boundaries.

3.2.3 3. Post-Execution Hook (Context & Exfiltration Prevention)

- **Software Engineering:** Role-Based Access Control (RBAC) scoping and provenance tagging ledger.

- **Applied Mathematics:** Differential privacy via Laplacian noise injection:

$$\tilde{y} = y + \text{Lap}\left(0, \frac{\Delta f}{\epsilon}\right) \tag{3.3}$$

and Proportional-Integral-Derivative (PID) cybernetic rate limiting for risk token-bucket bandwidth governors.

Chapter 4

Experiments

4.1 Experimental Setup and Testbed

Description of the testbed environment, simulated network topology, and agent execution sandbox.

4.2 Threat Models and Attack Scenarios

- Cache-Augmented Generation (CAG) memory poisoning scenarios.
- GraphRAG topological relationship injections.
- Steganographic image-based prompt injections (IPI).
- Multi-step Confused Deputy tool hijacking attacks.

4.3 Evaluation Datasets and Red-Teaming Benchmarks

- **Agent Red Teaming (ART) Benchmark:** Automated adversarial prompts targeting policy violations.
- **ExploitGym:** Autonomous tool exploitation and misuse benchmark.
- **Automated Fuzzing:** NVIDIA Garak and Microsoft PyRIT multi-turn conversational red-teaming.

4.4 Baseline Configurations

Comparison against unprotected agent baselines and traditional static prompt filter implementations.

Chapter 5

Evaluation

5.1 Detection Efficacy (RQ1 & RQ2)

Evaluation of Mahalanobis distance filters and 2D FFT spectral detectors against embedding anomalies and visual prompt injections.

5.2 Safety Barrier Enforcement (RQ3)

Empirical results of Control Barrier Function (CBF) boundary enforcement under multi-turn Confused Deputy attacks.

5.3 Cybernetic Rate Limiting Performance (RQ4)

Evaluation of the PID-governed token bucket against burst data exfiltration attempts.

5.4 System Overhead and Latency Analysis

Runtime latency, computational overhead, and throughput impact of the tri-hook middleware.

Chapter 6

Discussion

6.1 Balancing Mathematical Rigor and Software Practicality

Insights gained from mapping continuous mathematical boundary functions onto discrete agent API execution spaces.

6.2 Security Depth vs. Latency Overhead

Trade-off analysis between computational latency introduced at each hook and the corresponding theoretical security guarantees provided.

6.3 Assumptions and Limitations

Assumptions regarding embedding distributions, matrix inversion overhead in high dimensions, and boundary cases in tool hijacking defense.

Chapter 7

Future Work

7.1 Adaptive Latent Space Barrier Learning

Training parameterized neural barrier functions dynamically adapted to evolving agent toolspaces and contextual policies.

7.2 Hardware-Accelerated Cryptographic Hooks

Integrating Fully Homomorphic Encryption (FHE) with hardware acceleration for zero-knowledge runtime context queries.

7.3 Multi-Agent Swarm Defense Protocols

Extending tri-hook verification across decentralized multi-agent topologies and collective decision environments.

AI Disclosure

In accordance with academic integrity guidelines, generative AI tools and coding assistants were utilized during the preparation of this thesis for the following specific purposes:

- Grammar, style refinement, and drafting assistance for preliminary documentation.
- Syntax assistance for LaTeX templates, tables, and bibliographical formatting.

All core conceptual ideas, mathematical formulations, software architectural designs, experimental implementations, and analyses represent the original work of the author.

Bibliography

[1] OWASP Foundation, "OWASP Top 10 for Agentic Applications 2026," *OWASP Security Guidance*, 2026.

[2] A. D. Ames, S. Coogan, M. Egerstedt, M. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *2019 18th European Control Conference (ECC)*, IEEE, 2019, pp. 3420–3431.

[3] P. C. Mahalanobis, "On the generalized distance in statistics," *Proceedings of the National Institute of Sciences of India*, vol. 2, no. 1, pp. 49–55, 1936.

[4] NVIDIA, "Garak: Generative AI Red-teaming & Assessment Kit," *Open Source Security Framework*, 2024.

[5] Microsoft, "Python Risk Identification Tool for generative AI (PyRIT)," *Microsoft AI Red Team*, 2024.

Appendix A

Mathematical Proofs

A.1 Forward Invariance of Control Barrier Functions

Let $\mathcal{C} = \{x \in \mathbb{R}^n : h(x) \geq 0\}$ denote the 0-superlevel set of a continuously differentiable function $h : \mathcal{D} \subset \mathbb{R}^n \to \mathbb{R}$. The function h is a Control Barrier Function (CBF) if there exists an extended class $\mathcal{K}_\infty$ function α such that for the control system $\dot{x} = f(x) + g(x)u$:

$$\sup_{u \in U} \left[L_f h(x) + L_g h(x) u \right] \geq -\alpha(h(x)), \quad \forall x \in \mathcal{D} \tag{A.1}$$

Any controller choosing $u \in K_{cbf}(x)$ renders the set $\mathcal{C}$ forward invariant, guaranteeing safety for all $t \geq 0$.

Appendix B

Architecture Configuration and Schemas

B.1 Middleware Configuration Schema

Sample configuration schema for deploying the NetWatch tri-hook runtime interception gateway.